Flask Mock Challenge - Cosmic Travel (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p4-mock-challenge-cosmic-challenge)  (https://github.com/learn-co-curriculum/python-p4-mock-challenge-cosmic-challenge/issues/new)

It is the year 2100 and you run an interplanetary space travel agency. You are building a website to book scientists on missions to other planets.

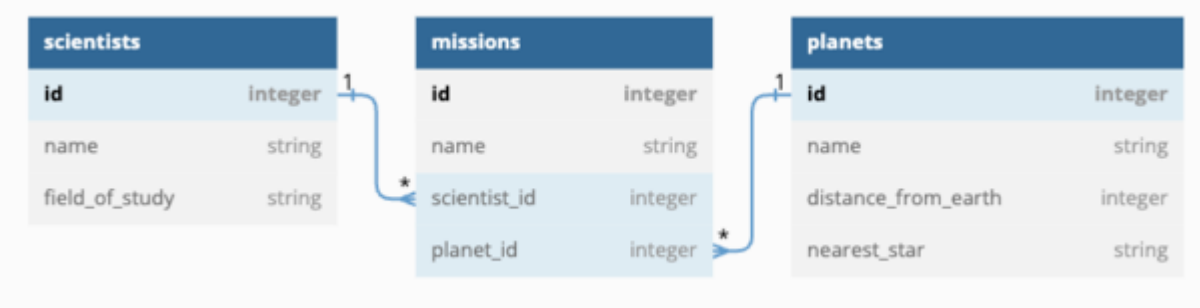
In this repo:

- There is a Flask application with some features built out.
- There is a fully built React frontend application.
- There are tests included which you can run using `pytest -x`.
- There is a file `mock-challenge-cosmic-challenge.postman_collection.json` that contains a Postman collection of requests for testing each route you will implement.

Depending on your preference, you can either check your API by:

- Using Postman to make requests
- Running `pytest -x` and seeing if your code passes the tests
- Running the React application in the browser and interacting with the API via the frontend

You can import `mock-challenge-cosmic-challenge.postman_collection.json` into Postman by pressing the `Import` button.



Select `Upload Files`, navigate to this repo folder, and select `mock-challenge-cosmic-challenge.postman_collection.json` as the file to import.

Setup

To download the dependencies for the frontend and backend, run:

```
pipenv install
pipenv shell
npm install --prefix client
```

You can run your Flask API on `localhost:5555` (http://localhost:5555) by running:

```
python server/app.py
```

You can run your React app on `localhost:4000` (http://localhost:4000) by running:

```
npm start --prefix client
```

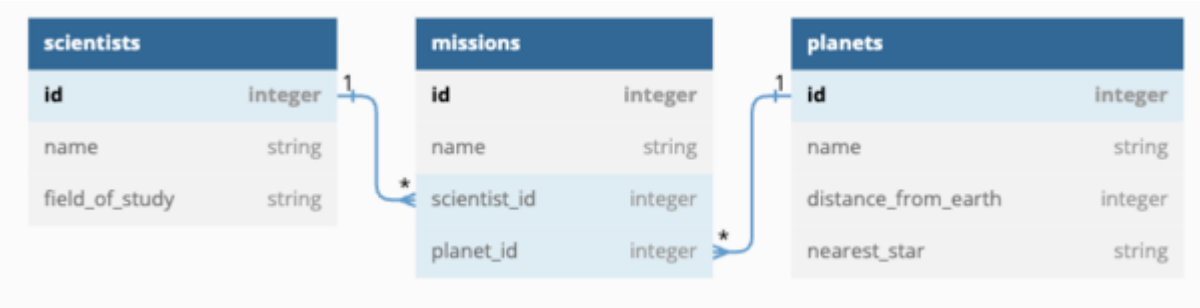
You are not being assessed on React, and you don't have to update any of the React code; the frontend code is available just so that you can test out the behavior of your API in a realistic setting.

Your job is to build out the Flask API to add the functionality described in the deliverables below.

Models

It is your job to build out Planet, Scientist, and Mission models so that scientists can book their missions. **In a given mission, one scientist will visit one planet**. Over their careers, **scientists will visit many planets** and **planets will be visited by many scientists**.

You will implement an API for the following data model:



The file `server/models.py` defines the model classes **without relationships**. Use the following commands to create the initial database `app.db`:

```
cd server
flask db init
flask db migrate -m 'initial model'
flask db upgrade head
```

Now you can implement the relationships as shown in the ER Diagram:

- A `Scientist` has (visits) many `Planets` through `Mission`s
- An `Planet` has (is visited by) many `Scientist`s through `Mission`s
- A `Mission` belongs to a `Scientist` and belongs to a `Planet`

Update `server/models.py` to establish the model relationships. Since a `Mission` belongs to a `Scientist` and a `Planet`, configure the model to cascade deletes.

Set serialization rules to limit the recursion depth.

Run the migrations and seed the database:

```
flask db migrate -m 'implement relationships'
flask db upgrade head
python seed.py
```

> If you aren't able to get the provided seed file working, you are welcome to generate your own seed data to test the application.

Validations

Add validations to the `Scientist` model:

- must have a `name`, and a `field_of_study`

Add validations to the `Mission` model:

- must have a `name`, a `scientist_id` and a `planet_id`

Routes

Set up the following routes. Make sure to return JSON data in the format specified along with the appropriate HTTP verb.

Recall you can specify fields to include or exclude when serializing a model instance to a dictionary using to_dict() (don't forget the comma if specifying a single field).

NOTE: If you choose to implement a Flask-RESTful app, you need to add code to instantiate the `Api` class in server/app.py.

GET /scientists

Return JSON data in the format below. **Note**: you should return a JSON response in this format, without any additional nested data related to each scientist.

```
[
  {
    "id": 1,
    "name": "Mel T. Valent",
    "field_of_study": "xenobiology"
  },
  {
    "id": 2,
    "name": "P. Legrange",
    "field_of_study": "orbital mechanics"
  }
]
```


GET /scientists/int:id

If the `Scientist` exists, return JSON data in the format below. Make sure to include a list of missions for the scientist.


```
"field_of_study": "Orbits",
    "id": 1,
    "name": "Joseph Richard",
    "missions": [
        {
            "id": 1,
            "name": "Explore Planet X.",
            "planet": {
                "distance_from_earth": 302613474,
                "id": 8,
                "name": "X",
                "nearest_star": "Shiny Star"
            },
            "planet_id": 8,
            "scientist_id": 1
```

```
        },
        {
            "id": 10,
            "name": "Explore Planet Y.",
            "planet": {
                "distance_from_earth": 1735242898,
                "id": 14,
                "name": "Y",
                "nearest_star": "Dim Star"
            },
            "planet_id": 14,
            "scientist_id": 1
        }
    ]
}
```

If the `Scientist` does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "error": "Scientist not found"
}
```

POST /scientists

This route should create a new `Scientist`. It should accept an object with the following properties in the body of the request:

```
{
  "name": "Evan Horizon",
  "field_of_study": "astronavigation"
}
```

If the `Scientist` is created successfully, send back a response with the new `Scientist`:

```
{
  "id": 3,
  "name": "Evan Horizon",
  "field_of_study": "astronavigation",
  "missions": []
}
```

If the `Scientist` is **not** created successfully due to validation errors, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "errors": ["validation errors"]
}
```

PATCH /scientists/:id

This route should update an existing `Scientist`. It should accept an object with one or more of the following properties in the body of the request:

```
{
  "name": "Bevan Horizon",
  "field_of_study": "warp drive tech"
}
```

If the `Scientist` is updated successfully, send back a response with the updated `Scientist` and a 202 `accepted` status code:

```
{
  "id": 2,
  "name": "Bevan Horizon",
  "field_of_study": "warp drive tech",
```

```
  "missions": []
}
```

If the `Scientist` is **not** updated successfully, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "errors": ["validation errors"]
}
```

OR, given an invalid ID, the appropriate HTTP status code, and the following JSON:

```
{
  "error": "Scientist not found"
}
```

DELETE /scientists/int:id

If the `Scientist` exists, it should be removed from the database, along with any `Mission`s that are associated with it. If you did not set up your models to cascade deletes, you need to delete associated `Mission`s before the `Scientist` can be deleted.

After deleting the `Scientist`, return an *empty* response body, along with the appropriate HTTP status code.

If the `Scientist` does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "error": "Scientist not found"
}
```

GET /planets

Return JSON data in the format below. **Note**: you should return a JSON response in this format, without any additional nested data related to each planet.

```
[
  {
    "id": 1,
    "name": "TauCeti E",
    "distance_from_earth": 1234567,
    "nearest_star": "TauCeti"
  },
  {
    "id": 2,
    "name": "Maxxor",
    "distance_from_earth": 99887766,
    "nearest_star": "Canus Minor"
  }
]
```

POST /missions

This route should create a new `Missions`. It should accept an object with the following properties in the body of the request:

```
{
  "name": "Project Terraform",
  "scientist_id": 1,
  "planet_id": 2
}
```

If the `Mission` is created successfully, send back a response about the new mission:

```
{
  "id": 21,
  "name": "Project Terraform",
  "planet": {
    "distance_from_earth": 9037395591,
```

```
    "id": 2,
    "name": "Planet X",
    "nearest_star": "Krystal"
  },
  "planet_id": 2,
  "scientist": {
    "field_of_study": "Time travel.",
    "id": 1,
    "name": "Jeremy Oconnor"
  },
  "scientist_id": 1
}
```

If the `Mission` is **not** created successfully, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "errors": ["validation errors"]
}
```

(Optional FYI) React `useCallback` hook

The `ScientistDetail` component in the React app uses the `useCallback` hook to memoize the function that fetches a scientist by id. The scientist detail is fetched when the component initially renders, and is fetched again after updating the scientist detail. `useCallback` caches the function to avoid recreating it .

Resources

- **useCallback API (https://react.dev/reference/react/useCallback)**
- **Should you add useCallback everywhere? (https://react.dev/reference/react/useCallback#should-you-add-usecallback-everywhere)**

This tool needs to be loaded in a new browser window

Load Flask Mock Challenge - Cosmic Travel (CodeGrade) in a new window